

CSE 5095: Types and Programming Languages

E Syntax, Small-step Semantics and Static Semantics

Stefan Muller

Tuesday, Sept. 8

1 Syntax

We will be working with a small language called **E** consisting of integer and string expressions. The grammar below is in BNF (Backus-Naur Form). We use $e ::= A \mid B \mid \dots$ to mean that an expression (with metavariable e) can look like form A or B , and so on. Note that the definition is recursive: an expression can, for example, be an expression plus another expression.

e	$::=$	$\bar{n}$	Numbers
		$ $	$"s"$ Strings
		$ $	$e + e$ Addition
		$ $	$e \wedge e$ Concatenation
		$ $	$ e $ String Length

Examples of valid expressions:

- $\bar{1}$
- $\bar{1} + |"Hello" \wedge "World"|$
- $"Hello" + 5$ (Note: this isn't well-typed, but it's still syntactically valid! The syntax doesn't distinguish!)

Examples of invalid expressions:

- $\bar{1.5}$
- $\bar{1} * \bar{5}$
- Hello (No quotes)

Important note: You might notice one thing missing from the above that's important for specifying expressions: parentheses. (Of course, it's not that important for this language because the operators commute, but in general it will be.) This is a difference between *abstract syntax*, which is how we generally specify programming languages when writing the theory and *concrete syntax*, which gets into these lower-level details such as operator precedence and parenthesization. Concrete syntax is very important if we're, for example, writing a parser, but generally the details are very complicated and obscure the points we're trying to make about the language. So, we'll write abstract syntax, use standard conventions for operator precedence, add parentheses as necessary (even if they're not actually in the syntax) and specify in words if it's unclear.

When we're defining the semantics of **E** and you see the metavariable e in a judgment, it can be instantiated with anything from the grammar above. That means we don't need to worry about the semantics of syntactically invalid expressions like $\bar{1.5}$ —it's just not valid to instantiate judgments with these in the first place. Given the note above, we will, however, talk about the semantics of expressions like $(\bar{1} + \bar{2}) + \bar{3}$ (which might be different from the semantics of $\bar{1} + (\bar{2} + \bar{3})$)

2 Small-step operational semantics

Next, we look at the “small-step operational semantics” of $\mathbf{E}$, which describe how to evaluate programs. There are two judgments, $e \text{ val}$ meaning that e is a value and can’t step anymore, and $e \mapsto e'$ meaning that e steps to e' . The rules for these judgments are below. Note that in rules STEPADD and STEPLEN , when we do $n_1 + n_2$ or $|s|$ (as opposed to $\overline{n_1} + \overline{n_2}$ and $|\text{“}s\text{”}|$), these are actually taking the mathematical addition of two integers and the actual number of characters in a string literal. We just use the same symbols for the actual underlying operation and for the syntax of the programming language. Rules other than the first three are “search” rules that allow us to step subexpressions.

$$\begin{array}{c}
\overline{n} \text{ val} \quad (\text{VALNUM}) \qquad \overline{\text{“}s\text{”}} \text{ val} \quad (\text{VALSTRING}) \qquad \overline{\overline{n_1} + \overline{n_2} \mapsto \overline{n_1 + n_2}} \quad (\text{STEPADD}) \\
\\
\overline{\text{“}s_1\text{”} \wedge \text{“}s_2\text{”} \mapsto \text{“}s_1s_2\text{”}} \quad (\text{STEPCAT}) \qquad \overline{|\text{“}s\text{”}| \mapsto |s|} \quad (\text{STEPLEN}) \qquad \frac{e_1 \mapsto e'_1}{e_1 + e_2 \mapsto e'_1 + e_2} \quad (\text{STEPSEARCHADDDLEFT}) \\
\\
\frac{e_2 \mapsto e'_2}{\overline{n_1} + e_2 \mapsto \overline{n_1} + e'_2} \quad (\text{STEPSEARCHADDRIGHT}) \qquad \frac{e_1 \mapsto e'_1}{e_1 \wedge e_2 \mapsto e'_1 \wedge e_2} \quad (\text{STEPSEARCHCATLEFT}) \\
\\
\frac{e_2 \mapsto e'_2}{\text{“}s_1\text{”} \wedge e_2 \mapsto \text{“}s_1\text{”} \wedge e'_2} \quad (\text{STEPSEARCHCATRIGHT}) \qquad \frac{e \mapsto e'}{|e| \mapsto |e'|} \quad (\text{STEPSEARCHLEN})
\end{array}$$

This is a “left-to-right” semantics because we enforce that the left side of an addition or concatenation is stepped fully before we step the right. We could have done the opposite and made a “right-to-left” semantics (or not specified an ordering and left the semantics nondeterministic).

Note that this means that we build a whole derivation using these rules to show that the program takes one step. For example:

$$\begin{array}{c}
\overline{\overline{1} + \overline{2} \mapsto \overline{3}} \quad (\text{STEPADD}) \\
\overline{(\overline{1} + \overline{2}) + \overline{3} \mapsto \overline{3} + \overline{3}} \quad (\text{STEPSEARCHADDDLEFT}) \\
\overline{\overline{0} + ((\overline{1} + \overline{2}) + \overline{3}) \mapsto \overline{0} + (\overline{3} + \overline{3})} \quad (\text{STEPSEARCHADDRIGHT})
\end{array}$$

And that’s just one step! The derivation generally consists of a bunch of the “search rules” at the bottom to find the one part of the expression that’s able to step now, and then one of the “actual” step rules to take the step.

We don’t usually write this full derivation, but instead just the steps:

$$\begin{array}{l}
\overline{0} + (\overline{1} + \overline{2}) + \overline{3} \\
\mapsto \overline{0} + (\overline{3} + \overline{3}) \\
\mapsto \overline{0} + \overline{6} \\
\mapsto \overline{6}
\end{array}$$

Another example:

$$\begin{array}{l}
|\text{“}ab\text{”} \wedge \text{“}c\text{”}| \\
\mapsto |\text{“}abc\text{”}| \\
\mapsto \overline{3}
\end{array}$$

Again, each of these steps is its own derivation but they’re pretty straightforward and mechanical to build. In fact, we often don’t even write all of the steps since a lot of them are kind of uninteresting. The judgment $e \mapsto^n e'$ means “ e steps to e' in n steps.”

$$\begin{array}{c}
\frac{}{e \mapsto^0 e} \quad (\text{STEPZERO}) \qquad \frac{e \mapsto e' \quad e' \mapsto^n e''}{e \mapsto^{n+1} e''} \quad (\text{STEPONE})
\end{array}$$

There's also a judgment $e \mapsto^* e'$ if we don't even care about the number of steps.

$$\frac{}{e \mapsto^* e} \text{ (STEPSTARZERO)} \qquad \frac{e \mapsto e' \quad e' \mapsto^* e''}{e \mapsto^* e''} \text{ (STEPSTARONE)}$$

The following theorem is obvious but technically necessary since these are different judgments. It's also a good warmup example of the types of things we'll prove about these judgments by rule induction.

Theorem 1. $e \mapsto^* e'$ if and only if $e \mapsto^n e'$ for some $n \geq 0$.

Proof. Forward direction ($e \mapsto^* e' \Rightarrow e \mapsto^n e'$):

By induction on the derivation of $e \mapsto^* e'$.

- Case STEPSTARZERO. Then $e = e'$. By STEPZERO, $e \mapsto^0 e$.
- Case STEPSTARONE. Then $e \mapsto e''$ and $e'' \mapsto^* e'$. By induction, $e'' \mapsto^n e'$ for some $n \geq 0$. By STEPONE, $e \mapsto^{n+1} e'$. We also have $n + 1 \geq 0$.

Reverse direction ($e \mapsto^n e' \Rightarrow e \mapsto^* e'$):

By induction on the derivation of $e \mapsto^n e'$.

- Case STEPZERO. Then $n = 0$ and $e = e'$. By STEPSTARZERO, $e \mapsto^* e$.
- Case STEPSTARONE. Then $n = m + 1$ and $e \mapsto e''$ and $e'' \mapsto^m e'$. By induction, $e'' \mapsto^* e'$. By STEPSTARONE, $e \mapsto^* e'$.

□

3 Static Semantics (Type System)

There are syntactically valid expressions that don't step. For example, $\bar{1} + \text{"Hello"}$. We say such expressions are “stuck.” There are also valid expressions that will step, but will (eventually) step to expressions that can no longer step (these will “get stuck”). For example, $(\bar{1} + \bar{2}) + \text{"Hello"} \mapsto \bar{3} + \text{"Hello"}$.

We fix this with a type system! We also call this the “static semantics” of the language because it's also a way of assigning meaning to a program (even just as simple as whether the program will evaluate properly) statically.

We have a syntax for types:

$$\begin{array}{ll} \tau ::= & \text{int} \quad \text{Integer type} \\ & | \text{string} \quad \text{String type} \end{array}$$

The typing judgment $e : \tau$ means “ e has type τ .”

$$\begin{array}{ccc} \frac{}{\bar{n} : \text{int}} \text{ (TYPENUM)} & \frac{}{\text{"s"} : \text{string}} \text{ (TYPESTRING)} & \frac{e_1 : \text{int} \quad e_2 : \text{int}}{e_1 + e_2 : \text{int}} \text{ (TYPEADD)} \\ \\ \frac{e_1 : \text{string} \quad e_2 : \text{string}}{e_1 \hat{\ } e_2 : \text{string}} \text{ (TYPECAT)} & & \frac{e : \text{string}}{|e| : \text{int}} \text{ (TYPELEN)} \end{array}$$

We can build derivations to show that expressions are well-typed (abbreviating TYPENUM as TN so it fits)

$$\frac{\frac{\frac{}{\bar{0} : \text{int}} \text{ (TN)}}{\bar{1} : \text{int}} \text{ (TN)} \quad \frac{\frac{\frac{}{\bar{2} : \text{int}} \text{ (TN)}}{\bar{1} + \bar{2} : \text{int}} \text{ (TYPEADD)} \quad \frac{\frac{}{\bar{3} : \text{int}} \text{ (TN)}}{(\bar{1} + \bar{2}) + \bar{3} : \text{int}} \text{ (TYPEADD)}}{\bar{0} + ((\bar{1} + \bar{2}) + \bar{3}) : \text{int}} \text{ (TYPEADD)}$$

This doesn't prevent us from writing stuck expressions: $\bar{1} + \text{"Hello"}$ is still syntactically valid (and will continue to be), but it doesn't have a type, that is, there is no τ such that $\bar{1} + \text{"Hello"} : \tau$. The idea is that if e is well-typed, that is, if $e : \tau$ for some τ , then e will never "get stuck". Next class, we'll formalize this as an idea called *type safety*.